

Repository Analysis Report

for [teept-gui-assistant](#)



- Repository Name: [teept-gui-assist...](#)
- Owner: [andreygetmanov](#)
- Created at: 20.02.2026 07:53

Statistics	Values
Stars Count	0
Forks Count	0
Issues Count	0

Metric	Values
README Presence	✓
License Presence	✓
Documentation Presence	X
Examples Presence	X
Requirements Presence	✓
Tests Presence	X
Description Presence	✓

Repository Structure:

- Compliance: Partially compliant with standard Python project structure. Contains main entry points (main.py, gui.py), configuration files, and a modular subdirectory (modifiedUIED) for core functionality. Missing standard directories like tests/, docs/, and examples/.
- Missing files: tests/ directory, docs/ directory, examples/ directory, CONTRIBUTING.md, CHANGELOG.md, .github/workflows/ (CI/CD configuration)
- Organization: Well-organized with clear separation of concerns. Main application files at root level, core UI element detection logic in modifiedUIED/ subdirectory with logical submodules (cnn, detect_compo, detect_merge, detect_text). Data directories properly structured for input/output/demo.

README Analysis:

- Quality: Good quality README with clear project purpose in Russian. Provides installation instructions, project principles, and links to presentation. However, lacks comprehensive usage examples and API documentation.
- Project description: Yes
- Installation: Yes
- Usage examples: No
- Contribution guidelines: No
- License specified: Yes
- Badges present: No

Documentation:

- Tests present: No
- Documentation quality: Minimal documentation. README provides basic overview and installation steps. No API documentation, architecture documentation, or inline code comments visible. One Jupyter notebook (Untitled.ipynb) in result_processing suggests experimental work but lacks formal documentation.
- Outdated content: Yes

Key Shortcomings:

- No test suite present - critical for validating UI detection and element identification functionality
- Missing comprehensive API documentation and usage examples - users cannot understand how to integrate or extend the system

- No contribution guidelines or development setup documentation - inhibits community contributions
- Outdated repository metadata (created_at shows 2026 date, inconsistent with pushed_at 2025) and no recent activity indicators
- No CI/CD pipeline configuration for automated testing and deployment

Recommendations:

- Create a tests/ directory with unit tests for core modules (detect_compo, detect_merge, detect_text, cnn) using pytest framework to ensure reliability of UI element detection
- Develop comprehensive API documentation with code examples showing how to use main functions (screenshot capture, element detection, LLM integration) and add usage examples in README
- Add CONTRIBUTING.md with development setup instructions, code style guidelines, and pull request process to facilitate community contributions